PROJECT PROPOSAL

LIDAR-Guided Hospital Navigation Robot

CSE 396 – Computer Engineering Project - Group 10

Autonomous Path-Following Robot with Obstacle Awareness

PROJECT MEMBERS

1) Mervan Deniz YILDIRIM

2) Ertuğrul ERGÜL

3) Sayed Khalilullah HASHIMI

4) Hicran KOÇ

5) Fatih Emre OĞAN

6) Muhammet Gökhan ÖZBEK

7) Samet ALKAN

8) Ömer Faruk GÜRSEL

1. Introduction

Hospitals are complex environments where patients and visitors frequently struggle to locate specific departments, wards or personal without assistance. Traditional static signage is often insufficient in large hospital buildings, and dedicated human guides are not always available.

This project proposes the development of a LiDAR-guided indoor navigation robot designed to assist people in finding their destination within a known floor plan. The robot operates on predefined paths mapped to a specific building floor, following a route from a start point to the visitor's desired destination. Navigation is handled through wheel odometry combined with IMU-based heading correction and periodic LiDAR scan-to-map matching.A key distinguishing feature of the system is reactive obstacle handling. The robot continuously monitors a forward-facing arc of its LiDAR scan for unexpected obstacles. If an obstacle is detected, the robot halts, waits for the obstacle to clear, and — if it persists — computes a short lateral detour using live LiDAR data before rejoining the predefined path. This behavior provides a practical, safe, and demonstrable level of autonomous intelligence without the full complexity of dynamic replanning.

2. Problem Statement

The specific problem this project addresses is:

> How can a low-cost autonomous robot reliably guide a person from an entry point to a specific destination within a known hospital environment, while safely handling unexpected dynamic obstacles along the path?

This problem encompasses several sub-challenges:

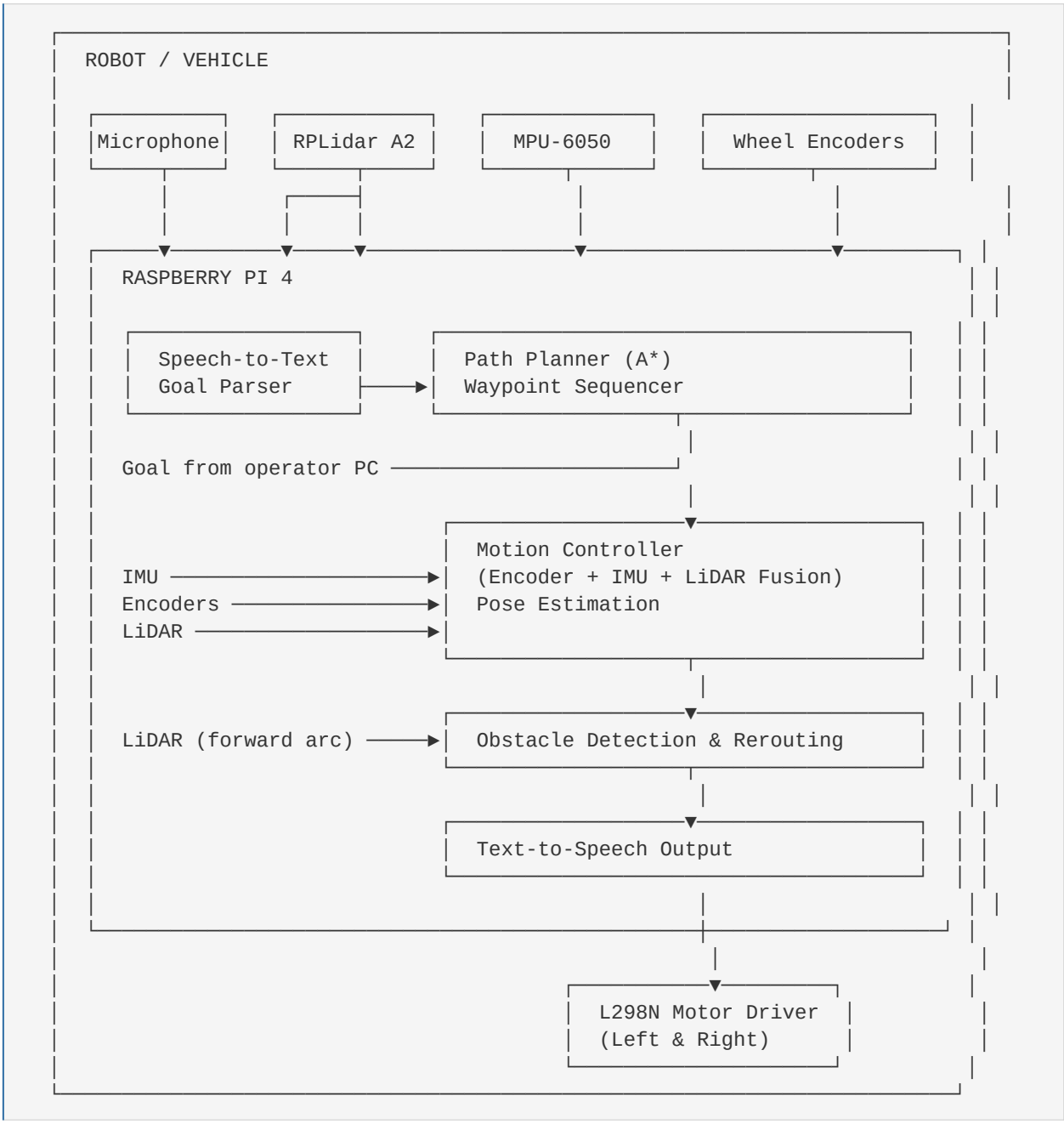
- Path execution without GPS or SLAM
- Drift compensation over short indoor routes (10–30 meters)
- Real-time obstacle detection and reactive behavior using a 2D LiDAR sensor
- Simple, accessible user interaction for destination selection
- Safe, reliable motor control and power management on a mobile platform
-

The scope is intentionally bounded to a single floor of a building with an area of 300–400 m^2, with a fixed, pre-mapped layout. This scope is realistic for a semester-length project while still producing a result that is genuinely useful and technically non-trivial.

3. Methodology

3.1 System Overview

The system is an autonomous indoor guide robot for hospital environments, built on a differential-drive platform. A Raspberry Pi 4 handles all computation: path planning, sensor fusion, obstacle detection, and voice I/O. Navigation goals are issued by voice command or from an operator computer; the robot provides spoken feedback throughout the mission. A digital twin in Gazebo is used for algorithm testing and scenario rehearsal during development.



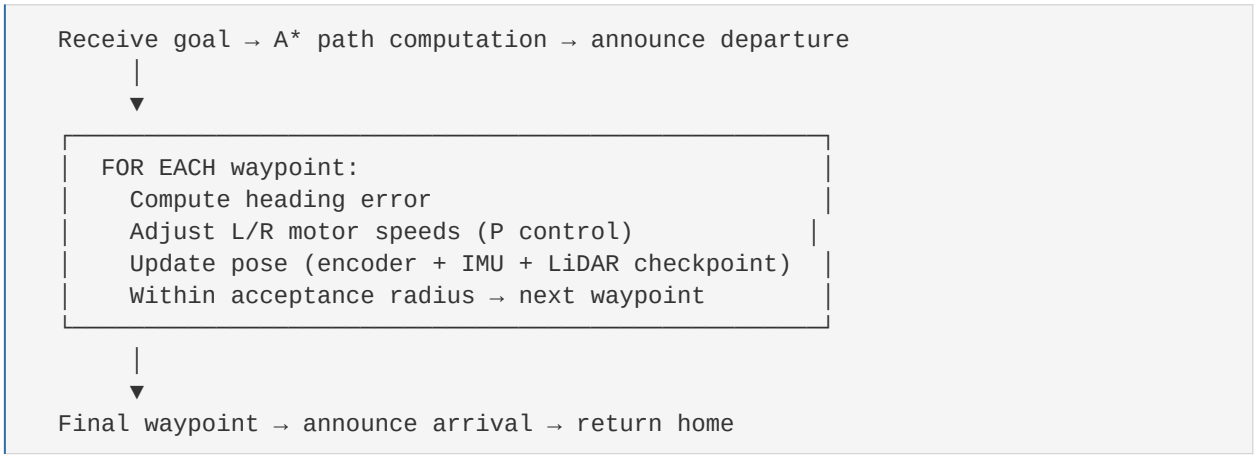
3.2 Localization Strategy

Localization is handled through three fused sources, with no GPS or full SLAM:

- **Wheel Odometry —** Encoder pulses per wheel are used to compute traveled distance and heading change via differential odometry equations, giving a continuous pose estimate (x, y, θ).
- **IMU Heading Fusion —** A complementary filter blends the encoder-derived heading with MPU-6050 gyroscope readings to reduce short-term noise and long-term bias drift.
- **LiDAR Scan Matching —** At predefined checkpoints (corridor junctions, room entrances), the live scan is correlated against a stored reference. On sufficient match, the pose estimate is snapped to the known checkpoint coordinates, resetting accumulated drift.

3.3 Path Planning and Execution

The building floor is represented as a pre-built 2D occupancy grid. Named destinations are registered as waypoints on this map. On receiving a goal, the robot runs A* to compute a collision-free waypoint sequence. A proportional heading controller steers toward each waypoint using the fused pose estimate, advancing to the next once within an acceptance radius (~10 cm). On completing the final waypoint the robot announces arrival and returns to its home position via the reverse path.



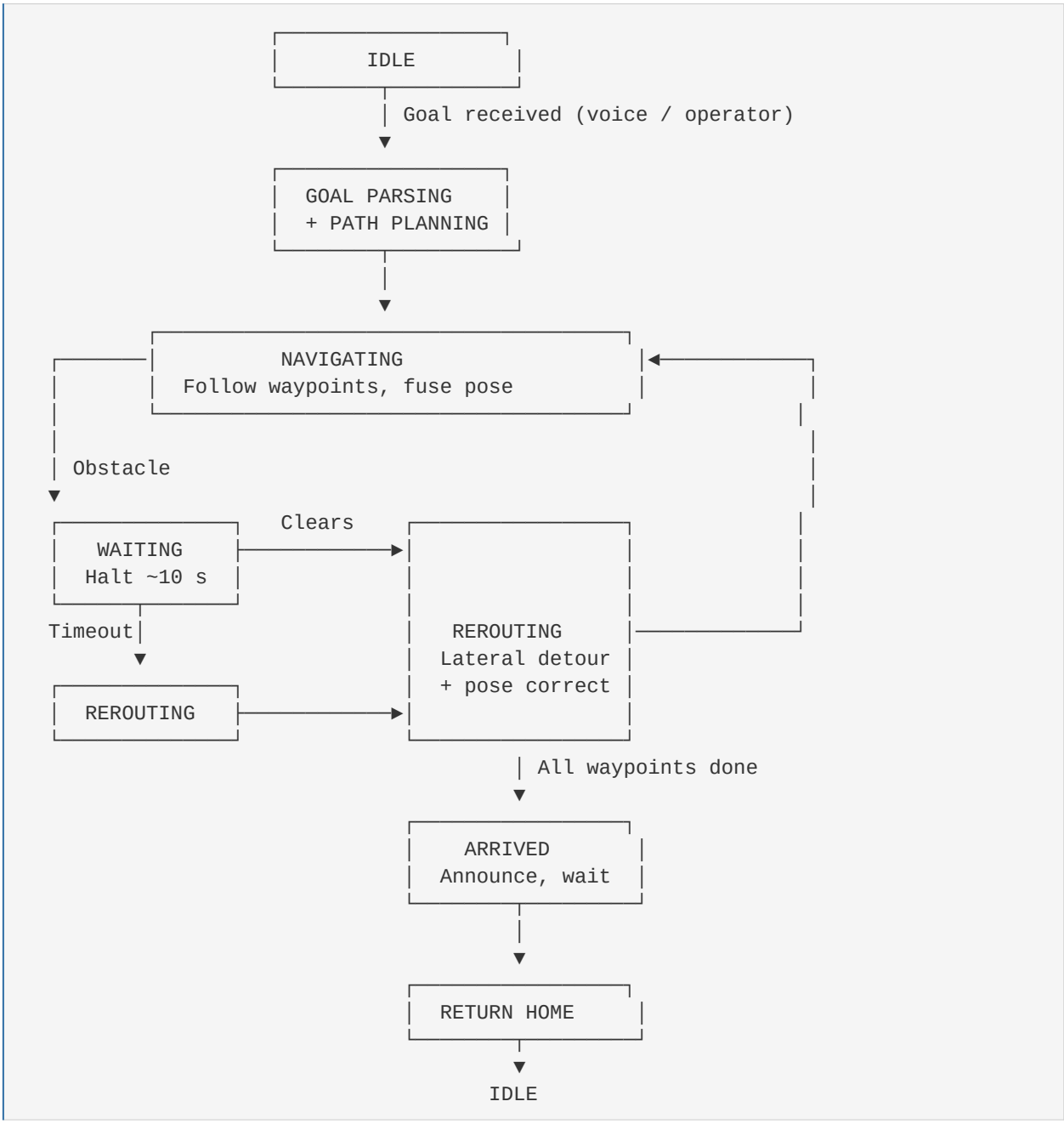
3.4 Obstacle Detection and Reactive Handling

The forward LiDAR arc (±25°) is monitored each control cycle. Any point closer than 50 cm triggers the state machine below.

State	Trigger Condition	Action
MOVING	Obstacle < 50 cm in forward arc	Halt, announce alert, start timer (~10–15 s)
WAITING	Obstacle clears before timeout	Resume MOVING
WAITING	Timeout expires, path still blocked	Transition to REROUTING
REROUTING	Scan left and right arcs	Select side with more free space, execute lateral detour (nudge → pass obstacle → return to path), verify pose via LiDAR checkpoint, resume MOVING

3.5 Scenario Flows

3.5.1 General Navigation Flow



3.5.2 Detailed Scenario — Hospital Visitor Guide

Step	Phase	Description
1	Goal Input	Visitor says "Take me to the MRI room." Speech-to-text maps the utterance to the registered waypoint. Robot confirms verbally.
2	Path Planning	A* computes a collision-free waypoint sequence from the current pose to the MRI room on the floor map.
3	Navigation	Robot follows the waypoint path; encoder + IMU fusion maintains pose, corrected at corridor-junction checkpoints via LiDAR scan matching.
4	Obstacle	A group blocks the corridor. Forward arc detects obstacle < 50 cm. Robot halts and waits (~10–15 s).
5a	Obstacle Clears	Path reopens; robot resumes original route.
5b	Rerouting	If still blocked at timeout, robot scans laterally, picks the clearer side, executes detour, re-anchors pose, and resumes.
6	Arrival	Final waypoint reached. Robot announces arrival and waits briefly.
7	Return	Robot navigates back to the entrance via the reverse path and returns to idle.

3.5.3 Movement Control Detail

```
── CONTROL LOOP (~50 ms) ──────────────────────────────────────
Read encoder pulses → compute Δdistance, Δheading
Read gyroscope → complementary filter → θ_fused
Update pose (x, y, θ)

At LiDAR checkpoint?
  Yes → correlate scan vs. reference map
         if match > threshold: snap pose to checkpoint

heading_error = angle_to(next_waypoint) − θ_fused
v_left  = base_speed − k × heading_error
v_right = base_speed + k × heading_error
→ Send to L298N motor driver

distance_to_waypoint < 10 cm?
  Yes → advance to next waypoint
────────────────────────────────────────────────────────────────
```

5. Techniques and Algorithms

#	Technique / Algorithm	Purpose	Implementation
01	**Odometry fusion** *Complementary filter*	Fuse wheel encoder ticks and IMU angular rate to produce a low-drift pose estimate — gyro handles fast turns, encoders handle straight-line distance	Encoders + IMU · Python `α=0.98 · real-time · RPi GPIO / I²C`
02	**SLAM with LiDAR** *KISS-ICP*	Build a real-time map and localise within it — odometry prior from encoders + IMU seeds each ICP step, reducing drift on long runs	KISS-ICP · RPLidar A1/A3 · Python / C++ `kiss-icp-ros2 · odom prior from row 01`
03	**Proportional path controller** *P-controller*	Convert waypoint heading error into differential motor commands — adjusts left/right wheel PWM proportionally to steer toward the next goal	Python · GPIO / PWM · RPi.GPIO `Kp-tunable · encoder velocity feedback`
04	**Navigation decision engine** *Behaviour tree / FSM*	Coordinate high-level states — idle, moving, avoiding, re-routing, goal reached — with composable, JSON-configurable behaviour modules	BehaviorTree.CPP · ROS2 Nav2 · py_trees `No global replanning · reactive fallback`
05	**Text-to-speech** *Voice communication (Turkish)*	Announce navigation events, obstacle warnings, and goal completions as audible speech — keeps the operator informed without needing a display	Piper TTS (tr_TR-dfki-medium) · Python `On-device · no network · ALSA / USB speaker`

5.Data

5.1 Map Data

The robot will operate on a predefined **2D map of the hospital floor**, which will be created by the project team through manual measurements and initial LiDAR scanning of the environment. This map represents walls and free navigation areas and will be used as the reference environment for the robot's path planning and navigation system.

5.2 Localization Data

The robot's position estimation will be obtained by combining data from the **IMU sensor and the LiDAR sensor**. The IMU provides motion and orientation information, while the LiDAR provides environmental distance measurements. By fusing these data sources, the system estimates the robot's position and orientation within the map.

5.3 Path and Destination Data

Navigation targets such as hospital departments will be represented as predefined **destination points on the map**. When the user selects a destination, the navigation system calculates a route from the robot's current position to the target location and updates the route if obstacles are detected.

5.4 Voice Command Data

User commands will be captured through a **USB microphone** and processed by a **Speech-to-Text system** running on the Raspberry Pi. The spoken command is converted into text and used to determine the selected destination within the hospital.

5.5 Runtime Sensor Data

During operation, the robot continuously processes **LiDAR scans for obstacle detection**, **IMU data for motion tracking**, and **voice command inputs** for user interaction. These data streams are used in real time by the navigation and control system.

6.Evaluation

6.1 Success Criteria

The success of the system will be evaluated based on the robot's ability to correctly interpret voice commands, navigate within the mapped environment using SLAM-based localization, and safely reach the selected destination while avoiding obstacles.

Level	Criterion	Metric
Must Have	Robot correctly recognizes destination from speech command	Destination selected correctly from speech input
Must Have	Robot navigates to the selected destination	Robot reaches the target location on the map
Must Have	Obstacle detection works correctly	Robot stops or re-plans when an obstacle appears
Must Have	Robot successfully avoids obstacles	Navigation continues and destination is reached
Should Have	SLAM localization remains stable during navigation	Robot position remains consistent on the map
Nice to Have	Digital twin visualization of robot position	Robot pose visualized in Gazebo or web dashboard
Nice to Have	Robot provides voice feedback to the user	Speaker announces navigation status

6.2 Evaluation Method

The system will be evaluated through navigation experiments in the mapped environment. In each test, a destination will be selected using the speech interface, and the robot will attempt to navigate to the target location using the SLAM-based navigation system. During the evaluation, the system will be observed to verify that the robot correctly interprets the spoken command, plans a route to the destination, detects obstacles, and safely reaches the target location. The robot's localization data and navigation results will be used to confirm successful operation.

7.Hardware and Budget

Component	Purpose	Unit Cost (USD)	Quantity	Total Cost (USD)
RPLidar A2	360° LiDAR sensor for environment scanning	~$200–250	1	~$200–250
Raspberry Pi 4 (2GB+)	Main computing unit running SLAM and navigation algorithms	~$35–50	1	~$35–50
L298N Motor Driver	Motor control for differential drive	~$3–5	1	~$3–5
DC Gear Motor	Drive motors for robot movement	~$5–8	2	~$10–16
Robot Wheels	Wheels attached to drive motors	~$2–4	2	~$4–8
Rotary Encoder	Wheel motion and distance measurement	~$2–4	2	~$4–8
MPU-6050 IMU	Orientation and motion sensing	~$1–3	1	~$1–3
LiPo Battery (11.1V, 2200mAh)	Power supply for the robot	~$12–18	1	~$12–18
Caster Wheel	Passive support wheel for balance	~$2–4	2	~$4–8
Robot Chassis	Mechanical frame of the robot	~$10–20	1	~$10–20
Youmi Mini USB 2.0 Microphone	Speech input for voice commands	~$5–8	1	~$5–8
MakerHawk Mini Speaker	Audio output for voice feedback	~$3–5	2	~$6–10
DROK PAM8406 Amplifier	Audio amplification for the speakers	~$5–7	1	~$5–7
Miscellaneous Components	Wires, connectors, mounts, and small hardware	~$5–10	1	~$5–10
Estimated Total Cost				~305 – 421 dolar

8.Risk Analysis

Risk	Likelihood	Impact	Mitigation
SLAM localization drift or temporary loss of localization	Medium	High	Use LiDAR-based SLAM with continuous map updates and perform testing in the target environment
Obstacle detection failure or delayed detection	Low	High	Tune LiDAR detection thresholds and perform obstacle testing during development
Speech recognition misinterprets user commands	Medium	Medium	Use limited command vocabulary and provide voice confirmation before navigation
Raspberry Pi USB instability with LiDAR	Low	High	Test USB connections early and use powered USB hub if required
Motor driver overheating or power instability	Low	Medium	Use proper heat dissipation and test power system under load
Changes in the environment (e.g., moved furniture) affecting navigation	Medium	Medium	Update the map or allow SLAM to adapt to small environmental changes
Integration delays between navigation and speech modules	Medium	Medium	Develop modules independently and integrate incrementally

9. Time Plan & Syllabus Milestones

Week	Phase / Milestone	Deliverable
W 1–3	Project Ideation & Structuring	Final project concept defined. Robot architecture designed. Required components selected (RPLidar A2, Raspberry Pi 4, encoders, IMU, and motor driver). Initial system plan prepared.
W 4 (15/3)	Project Proposal Submission	Proposal document submitted. Hardware orders placed and development environment preparation begins.
W 5	PC-Based Development & Hardware Setup	Raspberry Pi OS installed and configured. Python development environment prepared. Motor driver, encoder, and IMU initial tests performed.
W 6 (24/3)	Module Requirements (Headers)	Core software architecture finalized. Navigation, LiDAR processing, and obstacle detection module interfaces defined. Core codebase initialized and pushed to the repository.
W 7	Sub-system Integration	Sensor modules integrated with the motor control system. Robot begins executing basic movement and navigation tests using odometry data.
W 8 (7/4)	Module Documentation	Documentation of navigation algorithms, LiDAR data processing pipeline, and hardware wiring completed.
W 9	Navigation & Safety Testing	Robot performs waypoint navigation tests and obstacle avoidance behavior using LiDAR sensor data. Safety and stop-logic mechanisms validated.
W 10 (21/4)	Module Demonstrations	Individual system components (navigation, sensor processing, obstacle detection) demonstrated and validated through unit tests.
W 11–13	Full Integration & System Testing	All subsystems combined. End-to-end navigation tests performed in an indoor environment. Edge cases and stability issues debugged.
W 14 (19/5)	Final Project Demonstration	Final system demonstration video recorded. Robot completes a full navigation task including obstacle detection and path execution. Final report and documentation submitted.

10. Deliverables

At the end of the project quarter, the team expects to deliver the following:

Physical hardware: Working hospital navigation robot prototype

- Fully assembled mobile robot with RPLidar A2, Raspberry Pi 4, encoders, IMU, and motor driver
- Demonstrated navigation between predefined hospital locations (e.g., reception, room corridors, and service points)
- Demonstrated obstacle detection, waiting, and safe detour behavior in indoor environments
- Integrated voice interaction allowing users to communicate with the robot and request destinations

Software: Robot navigation and control system

- Open-source Python codebase including odometry, LiDAR processing, path planning, and obstacle avoidance logic
- Destination selection and navigation logic for guiding users to predefined hospital areas
- Voice communication module enabling simple spoken interaction between the user and the robot
- Digital Twin simulation of the robot and environment running on a PC for testing navigation logic and system behavior

Documentation: Project report and technical documentation

- Final written report covering introduction, system design, methods, results, discussion, and references
- System architecture diagram and hardware wiring schematic
- Description of navigation algorithms and robot control modules

Media: Demonstration video

- 3–5 minute video demonstrating a user interacting with the robot and the robot guiding the person to a selected hospital destination while handling obstacles

Module Structure and Task Distribution (8-Member Team)

The project is organized into 6 technical modules. The table below shows the role distribution across all 8 team members. Primary members lead and own their module; Secondary members contribute and provide cross-module support.

Person	M1 Voice Comm.	M2 Digital Twin	M3 HW & Motor	M4 SLAM	M5 NAV	M6 Sensor
GÖKHAN	**Primary**	Secondary	–	–	–	Secondary
HİCRAN	Secondary	–	**Primary**	Secondary	–	–
FATİH	Secondary	–	–	**Primary**	Secondary	–
SAMET	–	**Primary**	–	–	Secondary	–
MERVAN	–	Secondary	Secondary	**Primary**	–	–
ERTUĞRUL	–	–	Secondary	Secondary	**Primary**	–
SAYED	–	–	Secondary	–	–	**Primary**
ÖMER	–	–	–	–	**Primary**	Secondary

Primary = module lead and owner | Secondary = contributing support role | – = not assigned

Module Responsibilities

MODULE 1: Voice Communication (3 People)

Responsible for audio-based interaction between the robot and its environment. The module integrates a USB microphone (e.g., Youmi Mini USB 2.0 Microphone) for capturing ambient sound and spoken operator commands, processes the audio input to perform speech-to-text conversion, and produces spoken responses through a mini speaker (e.g., MakerHawk Mini Speaker) driven by an audio amplifier board (e.g., DROK 5W+5W Mini Audio Amplifier Board PAM8406). Handles distress sound detection and voice command parsing for operator control. Delivers a voice interface to other modules for triggering audio alerts and receiving transcribed commands.

MODULE 2: Digital Twin & Visualization (3 People)

Develops a real-time 3D visualization of the robot's state and surroundings using a simulation and visualization environment (e.g., Unity or Gazebo). Receives live telemetry (position, heading, sensor readings) from the robot and renders them on a digital floor map. Implements a priority-based pin-drop system to mark detected points of interest and reflect the robot's current status in the virtual environment.

MODULE 3: Hardware & Motor Control (4 People)

Handles all physical assembly and low-level hardware interfacing. Manages DC motor driving via PWM through the motor driver, encoder reading for odometry, and IMU (MPU-6050) integration for heading and stuck detection. Responsible for power distribution and UART communication between the microcontroller and the main compute unit. Provides a motion API (forward, backward, turn, stop) to higher-level modules.

MODULE 4: SLAM – Mapping & Localization (4 People)

Implements the Simultaneous Localization and Mapping (SLAM) pipeline. Processes LiDAR scan data to build and update a 2D occupancy grid map of the environment in real time. Maintains the robot's estimated pose (x, y, θ) and provides a localization interface for the navigation module. Handles map correction to minimize drift over longer exploration runs.

MODULE 5: NAV – Path Planning & Control (4 People)

Responsible for high-level autonomous movement. Consumes the map and pose from the SLAM module to plan collision-free paths to target waypoints. Implements the path-following controller, obstacle avoidance logic, and recovery behaviors such as stuck detection and re-routing. Delivers a navigation interface used by other modules to command and monitor the robot's movement.

MODULE 6: Sensor Integration & Control (3 People)

Manages all onboard sensors beyond the primary LiDAR: ultrasonic range sensors, IR obstacle sensors, and any additional environmental sensors. Responsible for sensor placement decisions, calibration, noise filtering, and publishing clean sensor readings to a shared data bus. Provides a unified sensor API that other modules (navigation, SLAM, hardware) can query for obstacle proximity and environmental data.

Literature Review

Indoor Positioning and Guidance GPS limitations in indoor settings [1] have led to infrastructure-dependent alternatives like Wi-Fi, BLE, and UWB [2-5]. While these offer precision, robotic guides provide a more interactive solution for navigating complex environments like hospitals [16-18]. This project employs a low-cost, map-based dead-reckoning approach to achieve reliable guidance without expensive external infrastructure [6].

Localization and Sensor Fusion To counter the inherent drift in wheel odometry [7], fusing encoder data with IMU-based heading via complementary filters offers a computationally efficient correction [8, 9]. Instead of resource-intensive SLAM, this system utilizes 2D LiDAR for scan-to-map matching [10, 11]. This method ensures lightweight and accurate localization by aligning live scans with pre-stored environmental data [12].

Obstacle Avoidance Reactive navigation is essential for safe operation in structured corridors [13]. While complex algorithms like VFH+ exist [14], simplified reactive strategies—such as automated halting or lateral offsets—provide sufficient safety for constrained hardware [15]. This approach minimizes computational overhead while maintaining the reliability necessary for autonomous indoor guidance.

References

[1] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.

[2] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proc. IEEE INFOCOM*, 2000.

[3] R. Faragher and R. Harle, "Location fingerprinting with Bluetooth low energy beacons," *IEEE Journal on Selected Areas in Communications*, vol. 33, no. 11, pp. 2418–2428, 2014.

[4] S. Gezici et al., "Localization via ultra-wideband radios: A look at positioning aspects for future sensor networks," *IEEE Signal Processing Magazine*, vol. 22, no. 4, pp. 70–84, 2005.

[5] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[6] J. Borenstein, H. R. Everett, and L. Feng, *Navigating Mobile Robots: Systems and Techniques*. AK Peters, Ltd., 1996.

[7] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.

[8] S. Madgwick, A. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," in *IEEE Int. Conf. on Rehabilitation Robotics*, 2011.

[9] R. Mahony, T. Hamel, and J. M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008.

[10] Hokuyo, "UTM-30LX Scanning Laser Rangefinder," Product Documentation, 2023.

[11] Slamtec, "RPLidar A2 Development Kit User Manual," Slamtec Co., Ltd., 2023.

[12] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," *IEEE Trans. on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.

[13] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on*

Robotics and Automation, vol. 2, no. 1, pp. 14–23, 1986.
[14] I. Ulrich and J. Borenstein, "VFH+: Reliable obstacle avoidance for fast mobile robots," in *Proc. IEEE Int. Conf. on Robotics and Automation*, 1998.
[15] J. Minguez and L. Montano, "Nearness diagram (ND) navigation: Collision avoidance in troublesome scenarios," *IEEE Transactions on Robotics and Automation*, vol. 20, no. 1, pp. 45–59, 2004.
[16] J. R. Carpman and M. A. Grant, *Design That Cares: Planning Health Facilities for Patients and Visitors*, 2nd ed. Jossey-Bass, 2002.
[17] F. Subhan et al., "Indoor positioning system: A survey," in *Int. Conf. on Communication Technologies*, 2019.
[18] Savioke, "Relay Robot: Autonomous Service Robot for Healthcare," Product Overview, 2022. [Online]. Available: https://www.savioke.com